

HDSP Research Group Laboratory Report

Practice No. 2

Applicant: Emanuel Navas Valencia

Tutor: Román Jacome

*School of Systems and Computer Engineering
Universidad Industrial de Santander*

September 3, 2026

Contents

1	Resume	2
2	From Linear Models to Neural Networks	2
2.1	Multilayer Perceptron	3
2.2	Training	3
2.2.1	How is back propagation used to compute nabla in an MLP?	3
2.3	Overfitting	4
2.3.1	What strategies can be used to prevent overfitting?	4
3	Convolutions and CNNs: Deep Learning that Respects Image Structure	5
3.1	Convolution	5
3.2	Common CNN Architectures	5
4	Inverse Problems and why they're Difficult	6
4.1	Definition	6
4.1.1	Why are many inverse problems ill-posed? In particular, why can multiple different x produce the same observation y under the model $y = Hx$?	7
4.2	Solving Inverse Problems	7
5	Implementation of PyTorch Exercises	7
5.1	Linear Layers	7
5.1.1	A1: Forward Pass of a Linear Layer	7
5.1.2	A2: Adding Nonlinearity	8
5.1.3	A3: Celsius to Fahrenheit	9
5.2	Multilayer Perceptrons	10
5.2.1	B1: MLP classification on vectorized images	10
5.2.2	B2: Overfitting demo	13
5.3	Convolutional Neural Networks	16
5.3.1	C1: CNN vs. MLP on image classification	16
5.3.2	C2: Inspect feature maps	18
5.3.3	C3: CNN for image deblurring	20
6	Guiding Questions	23
7	Conclusions	24

1 Resume

This report presents the roadmap and implementations for Practice No. 2 of the HDSP laboratory, which explores the transition from linear models to deep neural networks and their application to image processing and inverse problems. The theoretical foundations of perceptrons, multilayer perceptrons (MLPs), backpropagation, and overfitting mitigation strategies are reviewed, followed by an introduction to convolutional neural networks (CNNs) and their advantage in exploiting spatial structure through parameter sharing and locality. The mathematical formulation of inverse problems ($y = Hx + n$) is presented, along with their inherent ill-posedness, using denoising, deblurring, inpainting, and super-resolution as illustrative examples. These concepts are validated experimentally through a series of PyTorch exercises: implementing linear and nonlinear forward passes, fitting a Celsius-to-Fahrenheit regression, training an MLP classifier on FashionMNIST (achieving 87.38% test accuracy), demonstrating overfitting on a reduced training subset, and comparing an MLP against a CNN on MNIST classification, where the CNN outperformed the MLP (98.78% vs. 87.38% accuracy). Finally, a CNN-based deblurring model was trained and evaluated using PSNR and SSIM metrics, improving image quality from 14.54 dB / 0.47 (blurred) to 23.41 dB / 0.93 (reconstructed), confirming the effectiveness of deep learning approaches for solving inverse problems in imaging.

2 From Linear Models to Neural Networks

$$\hat{y} = w^T x + b$$

w is a weight vector and b is the bias.

Geometrically, it represents a hyperplane in the input space, and can only model linear relationships between inputs and outputs. The basic computational unit of these models is called a **perceptron**. **Perceptrons** compute a linear combination

$$z = w^T x + b$$

and then apply a non linear activation function

$$\phi(\cdot)$$

generating:

$$\hat{y} = \phi(z)$$

Without an activation function, stacking linear layers will still result in linear mapping (it does not increase the model's ability to express itself in a non-linear way).

An activation function affects the model's stability, convergence, and ability to capture complex structures in the data.

2.1 Multilayer Perceptron

A neural network consisting of multiple perceptrons, comprising an input layer, one or more hidden layers, and an output layer.

- The input layer receives the data x
- The hidden layers apply linear transformations and nonlinear activations in succession
- The output layer produces the final prediction

2.2 Training

The θ parameters (weights and biases) are updated to minimize the loss using the gradient descent method

$$\theta \leftarrow \theta - \eta \nabla_{\theta} * L$$

where η is the learning rate, a step size that controls how large each parameter update is in the direction of the negative gradient.

Computing ∇ using the chain rule is what constitutes backpropagation. In practice, training is organized into epochs: each epoch corresponds to a complete pass through the training data.

2.2.1 How is back propagation used to compute nabla in an MLP?

In an MLP, back propagation is used to compute the gradient of the loss function respect to all of the parameters of

$$\nabla_{\theta} L$$

where θ represents weights and biases and L is the loss function.

1. Forward pass:

For each layer l :

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

$$a^{(l)} = f(z^{(l)})$$

where w is the weight matrix, b is the bias vector, z is the input before activation, a is the output, and f is the activation function finally, we compute the loss:

$$L = L(a^{(L)}, y)$$

2. Compute the error at the output:

Back propagation starts at the last layer calculating

$$\delta^{(L)} = \frac{\partial L}{\partial z^{(L)}}$$

using the chain rule, this error shows how loss changes when changing the input of the last layer.

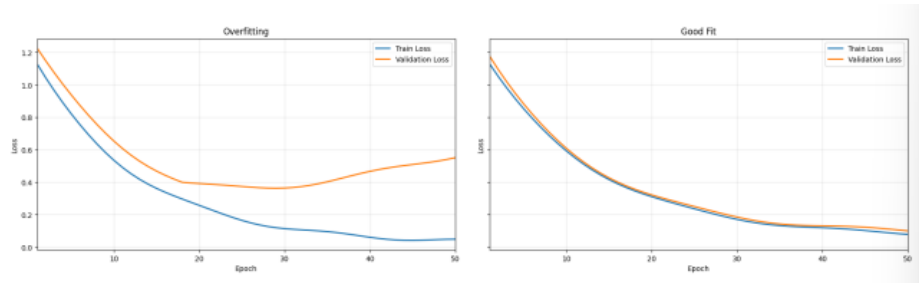


Figure 1: Overfitting vs Good Fit

3. Back propagate the error:

1. Take the error of the next layer
2. Back propagate it by changing the weights
3. Multiply it by the derivative of the activation function.

4. Get parameter gradients Once computed $\delta^{(l)}$ we obtain both the derivative of the loss in relation with the weight, and the loss in relation to the bias, with the metrics we can finally obtain $\nabla_{\theta} L$ and then an algorithm (like gradient descent) updates the parameters.

2.3 Overfitting

The training loss continues to decrease while the validation loss increases; this indicates that the model is not training properly, but rather is memorizing the training data, capturing patterns that are too specific and do not generalize well, causing it to perform poorly on new data.

2.3.1 What strategies can be used to prevent overfitting?

- Increasing the amount of training data.
- Augment the existent data: creating variations.
- Regularization (L1/L2): L1 adds a penalty based on the square of the weights, this makes the model prefer small weights. L2 adds a penalty based on the absolute value of the weights, this can make some weights become zero.
- Dropout: deactivate randomly chosen neurons during training.
- Early stopping: stopping the training process as soon as validation performance stops increasing.
- Reducing model complexity: decreasing layers or neurons if the model is too complex.
- Divide training and validation data: it helps detect if the model memorizes the training data.

3 Convolutions and CNNs: Deep Learning that Respects Image Structure

Convolutional neural networks were developed to address a limitation of neural networks when working with images: CNNs can take advantage of local structure and geometric regularities in images.

A grayscale image is a matrix where each $x_{i,j}$ represents an intensity at a specific location.

A color image is a tensor.

CNNs are designed to preserve spatial structure, something that cannot be achieved by flattening image vectors.

3.1 Convolution

A convolution applies a small kernel across the image:

$$(x * k)(i, j) = \sum x(i - m, j - n)k(m, n)$$

Where x denotes the input image, k is the convolution filter or kernel, (i, j) indicates the spatial location of the output pixel, and the indices m and n iterate over the size of the kernel.

The kernel iterates over the image and computes local weighted sums; each kernel detects different patterns, such as edges, textures, or corners.

A CNN block consists of

$$\text{convolution} \rightarrow \text{activation} \rightarrow \text{pooling/normalization}$$

Convolution kernels are the main parameters to be trained; normalization layers may also include parameters such as scale or shift, while pooling layers do not introduce new parameters.

Convolutions:

- Share parameters: the same kernel (set of weights) is used at all spatial locations in the image, which reduces the number of parameters and makes the model more efficient and less likely to suffer from overfitting.
- Take advantage of locality: each output value depends only on a small neighborhood of pixels; early layers learn simple local patterns such as edges, corners, or textures, while deep layers combine these features into more complex structures.

3.2 Common CNN Architectures

Practical CNNs are created by stacking multiple layers of convolutional blocks in a structured manner. Each kernel produces its own output feature map; stacking these yields the layer's output tensor. The number of kernels (also called output channels) is an architectural design decision and determines how many types of patterns the layer can learn to detect.

- Increasing the number of channels: applying more kernels allows the network to extract a greater variety of features.
- Downsampling: reducing the resolution allows the network to aggregate more information about larger regions of the image.

These two operations enable the network to construct hierarchical representations, ranging from simple local patterns to more abstract features.

The autoencoder architecture is an application of this: an encoder compresses the input into a high-level representation, while a decoder restores spatial resolution to produce an output the same size as the input.

4 Inverse Problems and why they're Difficult

4.1 Definition

A family of image-based tasks can be written as:

$$y = Hx + n$$

where x is the unknown ground truth, H is the forward operator that describes how the features are formed, n is the noise, and y is the observed value. The goal is to estimate x from y .

Common Inverse Problems

Forward Model Problem	Forward Model	Description
Denoising	$y = x + n$ ($H = I$)	The measurement is equal to the clean image plus noise; the <i>forward operator</i> is the identity.
Deblurring	$y = x * k + n$	The <i>forward operator</i> is a convolution with a blur kernel k . High-frequency details are attenuated, and edges become smoother.
Inpainting	$y = Hx + n$	The <i>forward operator</i> is a masking matrix that removes pixels.
Super-resolution	$y = D(Bx) + n$	The operator includes blur B followed by down-sampling D . The objective is to recover a high-resolution image from a low-resolution measurement.

Table 1: Common inverse problems and their forward models.

Most inverse problems are poorly formulated; several different x values can produce similar or exactly the same y values, and noise makes the problem worse, because a small perturbation in y can lead to enormous changes in a naive reconstruction.

4.1.1 Why are many inverse problems ill-posed? In particular, why can multiple different x produce the same observation y under the model $y = Hx$?

Many inverse problems are ill-posed because the forward operator H can lose information. In particular, multiple different values of x can produce the same observation y when H is not one-to-one (not injective). Mathematically, if H has a nontrivial null space, there can be a nonzero vector z such that $Hx = 0$. Then

$$H(x + z) = Hx + Hz = Hx$$

so both x and $x + z$ produce the same observation y . Therefore y does not contain enough information to uniquely determine the original x . For example, in image inpainting, the forward model removes or masks some pixels. Many different original images can have exactly the same visible pixels, so they produce the same observation. The missing pixels cannot be uniquely recovered without additional assumptions or prior information about the image.

4.2 Solving Inverse Problems

Where we observe y and want to recover x . One way to solve this problem is with deep learning, by training a neural network to learn a direct mapping from measurements to reconstructions:

$$\hat{x} = f_{\theta}(y)$$

This uses the same concepts introduced earlier: layers with parameters, nonlinear activations, and training by minimizing a discrepancy between the network's output $\hat{x}$ and the desired output x .

5 Implementation of PyTorch Exercises

5.1 Linear Layers

5.1.1 A1: Forward Pass of a Linear Layer

Report tensor shapes and verify that your implementation matches $X@w + b$.

Listing 1: Implementation of A1

```
1 import torch
2
3 def linear_forward(X, w, b):
4     return X @ w + b
5
6 B, d = 8, 5
7 X = torch.randn(B, d)
8 w = torch.randn(d)
```

```

9  b = torch.randn(1)
10
11 y_hat = linear_forward(X, w, b)
12 print("X:", X.shape, "w:", w.shape, "b:", b.shape, "y_hat:", y_hat.shape)

```

The reported tensor shapes are:

Tensor	Variable	Shape
Input matrix	X	(8, 5)
Weight vector	w	(5,)
Bias	b	(1,)
Output	y	(8,)

The implementation correctly matches $X@w + b$. Since X has shape (8, 5) and w has shape (5,), the matrix-vector multiplication $X@w$ produces an output of shape (8,). The bias b , with shape (1,), is then broadcast across the 8 output elements. Therefore, the final prediction y has shape (8,), confirming that the implementation correctly computes $X@w + b$.

5.1.2 A2: Adding Nonlinearity

Show a small histogram or min/max summary for outputs with two different activations and explain the difference in one sentence.

Listing 2: Implementation of A2

```

1  import torch
2
3  def relu(t):
4      return torch.clamp(t, min=0.0)
5
6  def sigmoid(t):
7      return 1.0/(1.0 + torch.exp(-t))
8
9  def perceptron_forward(X, w, b, activation):
10     z = X @ w + b
11     y_hat = activation(z)
12     return y_hat
13
14 y_relu = perceptron_forward(X, w, b, relu)
15 y_sig = perceptron_forward(X, w, b, sigmoid)
16 print("relu:", y_relu.min().item(), y_relu.max().item())
17 print("sigmoid:", y_sig.min().item(), y_sig.max().item())

```

The output ranges for the two activation functions are summarized below:

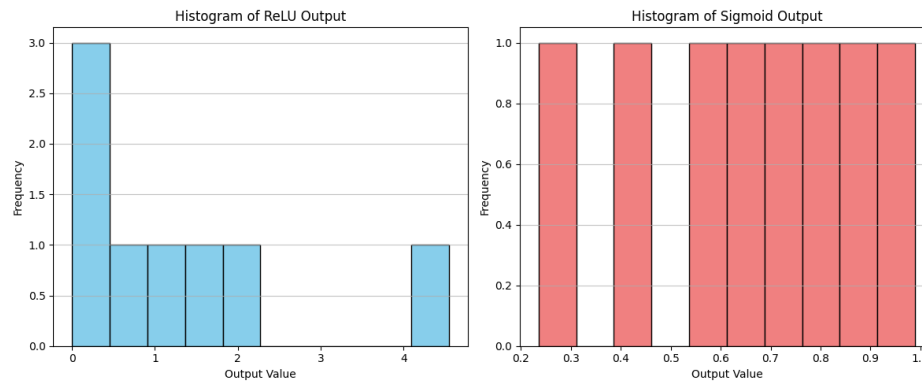


Figure 2: Histogram of the output value frequencies

Activation	Minimum Output	Maximum Output
ReLU	0.0000	4.5449
Sigmoid	0.2351	0.9895

The results show that ReLU sets negative values to zero and can produce unbounded positive outputs, whereas the sigmoid function maps all outputs to values between 0 and 1.

5.1.3 A3: Celsius to Fahrenheit

Plot the loss vs. iteration and report the learned (w, b) compared to (w_{true}, b_{true})

Listing 3: Implementation of A3

```

1 import torch
2 import matplotlib.pyplot as plt
3
4 torch.manual_seed(42)
5 B = 64
6 x_c = 40*torch.rand(B, 1) - 10
7 w_true = torch.tensor([[9/5]])
8 b_true = torch.tensor([32.0])
9 y_f = x_c @ w_true + b_true + 0.5*torch.randn(B, 1)
10
11 w = torch.randn(1, 1, requires_grad=True)
12 b = torch.randn(1, requires_grad=True)
13
14 iterations = 500
15 lr = 3.8e-3
16
17 loss_history = []
18
19 for it in range(iterations):

```

```

20     y_hat = x_c @ w + b
21     loss = ((y_hat - y_f)**2).mean()
22     loss.backward()
23     with torch.no_grad():
24         w -= lr * w.grad
25         b -= lr * b.grad
26     w.grad.zero_()
27     b.grad.zero_()
28     loss_history.append(loss.item())
29
30     if it % 100 == 0:
31         print(it, loss.item(), w.item(), b.item())
32
33 # Outputs
34 plt.figure(figsize=(8, 5))
35 plt.plot(range(iterations), loss_history, label='Training Loss')
36 plt.title('Loss vs. Iteration')
37 plt.xlabel('Iteration')
38 plt.ylabel('Loss')
39 plt.grid(True)
40 plt.legend()
41 plt.show()
42
43 print(f"\nTrue w: {w_true.item():.4f}, True b: {b_true.item():.4f}")
44 print(f"Learned w: {w.item():.4f}, Learned b: {b.item():.4f}")

```

The true and learned parameter values are:

$$\begin{aligned}
 w_{\text{true}} &= 1.8000, & b_{\text{true}} &= 32.0000, \\
 w_{\text{learned}} &= 2.0178, & b_{\text{learned}} &= 27.0103.
 \end{aligned}$$

The learned parameters are close to the true values, with the small differences attributable to the noise added to the training data and the optimization process.

5.2 Multilayer Perceptrons

5.2.1 B1: MLP classification on vectorized images

We're given the FashionMNIST dataset, that contains grayscale images of clothing and 10 categories (shirt, pants, etc). The goal is to train a model to correctly classify these images into their corresponding classes.

Listing 4: Implementation of B1

```

1 import torch
2 from torchvision import datasets, transforms
3 from torch.utils.data import DataLoader

```

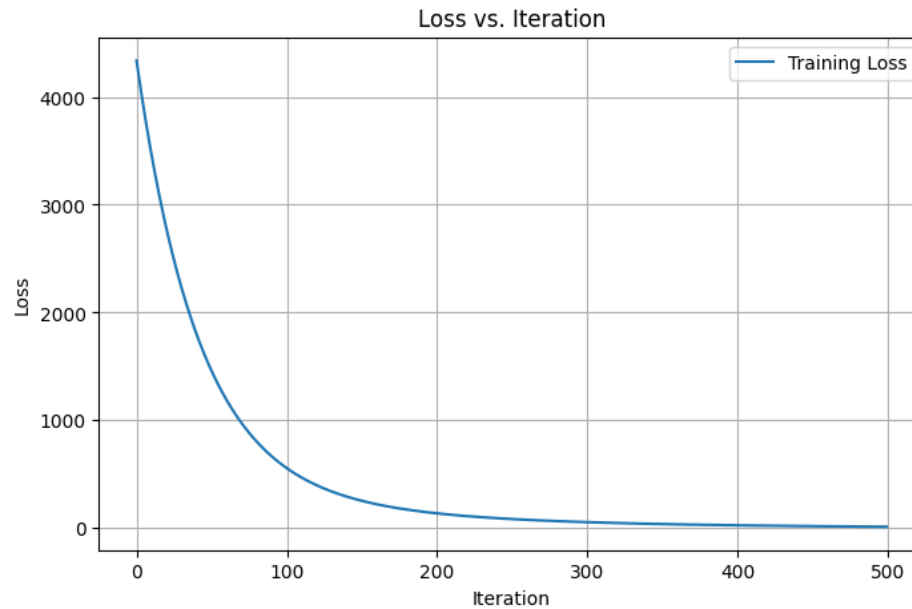


Figure 3: Plot for the code output

```
4 from sklearn.metrics import confusion_matrix
5 import numpy as np
6
7 tfm = transforms.Compose([transforms.ToTensor()])
8 train_ds = datasets.FashionMNIST("./data", train=True, download=True,
9     transform=tfm)
10 test_ds = datasets.FashionMNIST("./data", train=False, download=True,
11     transform=tfm)
12
13 train_dl = DataLoader(train_ds, batch_size=128, shuffle=True)
14 test_dl = DataLoader(test_ds, batch_size=256, shuffle=False)
15
16 class MLPClassifier(torch.nn.Module):
17     def __init__(self, input_value_amount=28*28, hidden_layer_neuron_amount=256,
18         number_of_classes=10):
19         super().__init__()
20         self.net = torch.nn.Sequential(
21             torch.nn.Linear(input_value_amount, hidden_layer_neuron_amount),
22             torch.nn.ReLU(),
23             torch.nn.Linear(hidden_layer_neuron_amount, number_of_classes)
24         )
25
26     def forward(self, x):
27         x = x.view(x.size(0), -1)
28         return self.net(x)
```

```
26 '''
27
28 def train_classifier(model, loader, lr=1e-3, epochs=5):
29     optimizer = torch.optim.Adam(model.parameters(), lr=lr)
30     ce = torch.nn.CrossEntropyLoss()
31
32     '''
33     model.train()
34
35     for epoch in range(epochs):
36         running_loss = 0.0
37
38         for i, (xb, yb) in enumerate(loader):
39             optimizer.zero_grad()
40             logits = model(xb)
41             loss = ce(logits, yb)
42             loss.backward()
43             optimizer.step()
44             running_loss += loss.item()
45
46         print(f"Epoch {epoch+1}/{epochs}, Loss: {running_loss/len(loader):.4f}")
47
48     return model
49     '''
50
51 def accuracy(model, loader):
52     correct, total = 0, 0
53     model.eval()
54
55     '''
56     with torch.no_grad():
57         for xb, yb in loader:
58             pred = torch.argmax(model(xb), dim=1)
59             correct += (pred == yb).sum().item()
60             total += yb.numel()
61
62     return correct/total
63     '''
64
65 model = MLPClassifier()
66 model = train_classifier(model, train_dl, lr=1e-3, epochs=5)
67 print("test accuracy:", accuracy(model, test_dl))
68
69 all_preds = []
70 all_labels = []
71
```

```

72 model.eval()
73
74 with torch.no_grad():
75     for xb, yb in test_dl:
76         logits = model(xb)
77         preds = torch.argmax(logits, dim=1)
78         all_preds.extend(preds.cpu().numpy())
79         all_labels.extend(yb.cpu().numpy())
80
81 cm = confusion_matrix(all_labels, all_preds)
82 print("\nConfusion Matrix:")
83 print(cm)

```

Test Accuracy = 0.8738 (1)

$$C = \begin{bmatrix} 812 & 0 & 6 & 34 & 4 & 1 & 128 & 0 & 15 & 0 \\ 3 & 956 & 1 & 28 & 6 & 0 & 4 & 0 & 2 & 0 \\ 10 & 0 & 721 & 15 & 150 & 0 & 97 & 0 & 7 & 0 \\ 19 & 3 & 5 & 906 & 27 & 1 & 33 & 0 & 6 & 0 \\ 0 & 0 & 60 & 37 & 837 & 0 & 61 & 0 & 5 & 0 \\ 0 & 0 & 0 & 1 & 0 & 960 & 0 & 29 & 2 & 8 \\ 116 & 1 & 64 & 35 & 76 & 0 & 689 & 0 & 19 & 0 \\ 0 & 0 & 0 & 0 & 0 & 29 & 0 & 949 & 0 & 22 \\ 3 & 0 & 1 & 3 & 5 & 4 & 8 & 5 & 971 & 0 \\ 0 & 0 & 0 & 0 & 0 & 15 & 1 & 47 & 0 & 937 \end{bmatrix}. \quad (2)$$

5.2.2 B2: Overfitting demo

Report test accuracy and include a confusion matrix for at least 5 classes.

Listing 5: Implementation of B2

```

1  '''python
2  import torch
3  from torch.utils.data import Subset, DataLoader
4  import numpy as np
5  import matplotlib.pyplot as plt
6
7  idx = np.random.choice(len(train_ds), size=1000, replace=False)
8  small_train_dl = DataLoader(
9      Subset(train_ds, idx),
10     batch_size=128,
11     shuffle=True
12 )
13

```

```
14 class MLPClassifier(torch.nn.Module):
15     def __init__(
16         self,
17         input_value_amount=28 * 28,
18         hidden_layer_neuron_amount=256,
19         number_of_classes=10
20     ):
21         super().__init__()
22         self.net = torch.nn.Sequential(
23             torch.nn.Linear(input_value_amount, hidden_layer_neuron_amount),
24             torch.nn.ReLU(),
25             torch.nn.Linear(hidden_layer_neuron_amount, number_of_classes)
26         )
27
28     def forward(self, x):
29         x = x.view(x.size(0), -1)
30         return self.net(x)
31
32 def train_with_curves(
33     model,
34     train_loader,
35     val_loader,
36     lr=1e-3,
37     epochs=10,
38     weight_decay=0.0
39 ):
40     optimizer = torch.optim.Adam(
41         model.parameters(),
42         lr=lr,
43         weight_decay=weight_decay
44     )
45     ce = torch.nn.CrossEntropyLoss()
46
47     train_losses = []
48     val_losses = []
49
50     for epoch in range(epochs):
51         model.train()
52         running_train_loss = 0.0
53
54         for xb, yb in train_loader:
55             optimizer.zero_grad()
56             logits = model(xb)
57             loss = ce(logits, yb)
58             loss.backward()
59             optimizer.step()
```

```

60         running_train_loss += loss.item()
61
62         train_losses.append(running_train_loss / len(train_loader))
63
64         model.eval()
65         running_val_loss = 0.0
66
67         with torch.no_grad():
68             for xb, yb in val_loader:
69                 logits = model(xb)
70                 running_val_loss += ce(logits, yb).item()
71
72         val_losses.append(running_val_loss / len(val_loader))
73
74         print(
75             f"Epoch {epoch + 1}/{epochs}, "
76             f"Train Loss: {train_losses[-1]:.4f}, "
77             f"Val Loss: {val_losses[-1]:.4f}"
78         )
79
80         return train_losses, val_losses
81
82 model = MLPClassifier()
83
84 train_losses, val_losses = train_with_curves(
85     model,
86     small_train_dl,
87     test_dl,
88     epochs=15
89 )
90
91 plt.figure(figsize=(10, 6))
92 plt.plot(train_losses, label="Training Loss")
93 plt.plot(val_losses, label="Validation Loss")
94 plt.title("Training and Validation Loss Curves")
95 plt.xlabel("Epoch")
96 plt.ylabel("Loss")
97 plt.legend()
98 plt.grid(True)
99 plt.show()
100 '''

```

$$\text{Test Accuracy} = 0.8021 \quad (3)$$

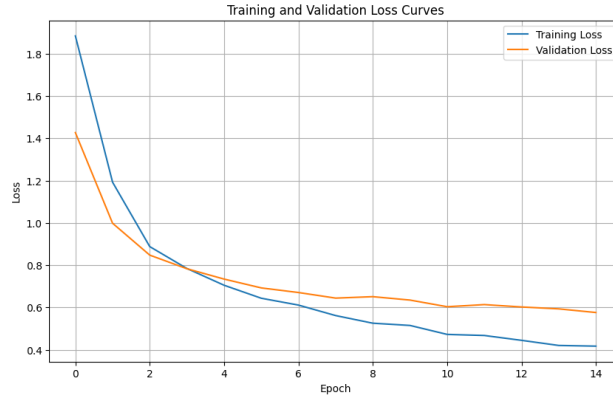


Figure 4: Validation Loss vs Training Loss

$$\mathbf{C}_{\text{overfit}} = \begin{bmatrix} 764 & 14 & 11 & 61 & 8 & 0 & 120 & 0 & 21 & 1 \\ 2 & 932 & 12 & 39 & 4 & 0 & 9 & 0 & 2 & 0 \\ 16 & 4 & 686 & 6 & 157 & 2 & 124 & 0 & 5 & 0 \\ 32 & 23 & 14 & 796 & 52 & 1 & 78 & 0 & 4 & 0 \\ 0 & 4 & 146 & 29 & 717 & 3 & 96 & 0 & 5 & 0 \\ 1 & 0 & 0 & 2 & 0 & 866 & 1 & 63 & 4 & 63 \\ 139 & 6 & 118 & 32 & 141 & 1 & 529 & 0 & 34 & 0 \\ 0 & 0 & 0 & 0 & 0 & 53 & 0 & 879 & 0 & 68 \\ 6 & 2 & 14 & 6 & 3 & 17 & 18 & 7 & 924 & 3 \\ 0 & 0 & 0 & 1 & 0 & 19 & 0 & 51 & 1 & 928 \end{bmatrix}. \quad (4)$$

As observed in the generated plot, the training loss consistently decreases over the epochs, indicating that the model is effectively learning to fit the dataset. However, the validation loss initially decreases but then starts to flatten out or even slightly increase after a certain point (around epoch 11-12 in this example), while the training loss continues to fall. This divergence between the training and validation loss curves is a classic sign of overfitting.

5.3 Convolutional Neural Networks

5.3.1 C1: CNN vs. MLP on image classification

Report CNN vs. MLP accuracy in a small table and write 2–3 sentences explaining the difference.

Listing 6: Implementation of C1

```

1 import torch
2 from torchvision import datasets, transforms
3 from torch.utils.data import DataLoader
4
5 tfm = transforms.Compose([transforms.ToTensor()])

```

```
6 train_ds = datasets.MNIST("./data", train=True, download=True, transform=tfm)
7 test_ds = datasets.MNIST("./data", train=False, download=True, transform=tfm)
8 train_dl = DataLoader(train_ds, batch_size=128, shuffle=True)
9 test_dl = DataLoader(test_ds, batch_size=256, shuffle=False)
10
11 class CNNModel(torch.nn.Module):
12     def __init__(self, n_classes=10):
13         super().__init__()
14         self.features = torch.nn.Sequential(
15             torch.nn.Conv2d(1, 16, 3, padding=1),
16             torch.nn.ReLU(),
17             torch.nn.MaxPool2d(2),
18             torch.nn.Conv2d(16, 32, 3, padding=1),
19             torch.nn.ReLU(),
20             torch.nn.MaxPool2d(2),
21         )
22         self.head = torch.nn.Sequential(
23             torch.nn.Flatten(),
24             torch.nn.Linear(32*7*7, 128),
25             torch.nn.ReLU(),
26             torch.nn.Linear(128, n_classes),
27         )
28     def forward(self, x):
29         x = self.features(x)
30         return self.head(x)
31
32 def train_classifier(model, loader, lr=1e-3, epochs=5):
33     optimizer = torch.optim.Adam(model.parameters(), lr=lr)
34     ce = torch.nn.CrossEntropyLoss()
35
36     model.train()
37     for epoch in range(epochs):
38         running_loss = 0.0
39         for i, (xb, yb) in enumerate(loader):
40             optimizer.zero_grad()
41             logits = model(xb)
42             loss = ce(logits, yb)
43             loss.backward()
44             optimizer.step()
45             running_loss += loss.item()
46         print(f"Epoch {epoch+1}/{epochs}, Loss: {running_loss/len(loader):.4f}")
47     return model
48
49 def accuracy(model, loader):
50     correct, total = 0, 0
51     model.eval()
```

```

52     with torch.no_grad():
53         for xb, yb in loader:
54             pred = torch.argmax(model(xb), dim=1)
55             correct += (pred == yb).sum().item()
56             total += yb.numel()
57         return correct/total
58
59 cnn = CNNModel()
60 cnn = train_classifier(cnn, train_dl, epochs=5)
61 print("CNN test accuracy:", accuracy(cnn, test_dl))

```

CNN vs. MLP Accuracy Comparison

Model	Test Accuracy
MLP	0.8738
CNN	0.9878

Table 2: Test accuracy comparison between MLP and CNN models.

The table clearly shows that the **Convolutional Neural Network (CNN)** achieved **significantly higher accuracy (0.9878)** compared to the **Multi-Layer Perceptron (MLP) (0.8738)** on the image classification task. This difference highlights the CNN's superior ability to capture spatial hierarchies and local patterns in image data through its convolutional and pooling layers. Unlike MLPs, which treat images as flattened vectors and lose spatial information, CNNs leverage this structure, leading to more robust and accurate feature extraction for tasks such as digit recognition.

5.3.2 C2: Inspect feature maps

Provide a figure showing multiple feature maps from the first convolutional layer and describe what you observe.

Listing 7: Implementation of C2

```

1  import torch
2  import matplotlib.pyplot as plt
3
4  x0, y0 = test_ds[0]
5  x0b = x0.unsqueeze(0)
6
7  cnn.eval()
8  with torch.no_grad():
9      feature_maps = cnn.features[0](x0b) # cnn.features[0] is the first Conv2d layer
10

```

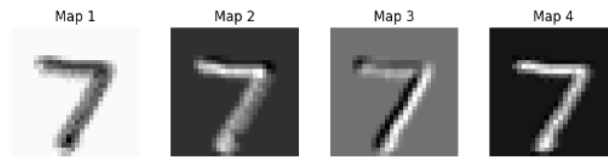


Figure 5: Feature maps 1 - 4

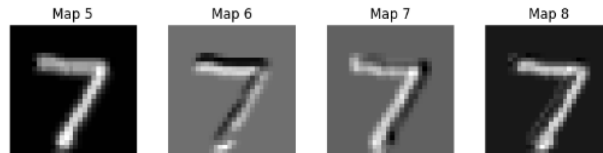


Figure 6: Feature maps 5 - 8

```

11 feature_maps_np = feature_maps.squeeze(0).cpu().numpy()
12
13 num_visualize = min(8, feature_maps_np.shape[0])
14
15 fig, axes = plt.subplots(1, num_visualize, figsize=(num_visualize * 2, 2))
16 if num_visualize == 1:
17     axes = [axes]
18
19 for i in range(num_visualize):
20     axes[i].imshow(feature_maps_np[i], cmap='gray')
21     axes[i].set_title(f'Map {i+1}')
22     axes[i].axis('off')
23
24 plt.tight_layout()
25 plt.show()

```

The displayed images are the activation maps from the first convolutional layer. Each image corresponds to one filter in that layer. These filters are essentially pattern detectors. When a filter 'sees' a pattern it was trained to detect in the input image, it produces a high activation (brighter areas in the grayscale image). Conversely, areas where the pattern is not present or is suppressed will have lower activations (darker areas).

Looking at these feature maps, you can observe that different filters respond to different aspects of the input digit '7'. Some maps might highlight edges, corners, or specific strokes (like the horizontal bar or the diagonal line of the '7'). This visual inspection confirms that the CNN's early layers are learning to extract basic, localized features from the raw pixel data, which are then combined by deeper layers to form more complex representations and ultimately classify the digit.

5.3.3 C3: CNN for image deblurring

Show examples of: ground truth x , blurred image y , CNN reconstruction $\hat{x}$. Compute PSNR and SSIM for the Blurred image vs. ground truth, and for the CNN reconstruction vs. ground truth.

Listing 8: Implementation of C3

```

1 import torch
2 import torch.nn.functional as F
3 from torchvision import datasets, transforms
4 from torch.utils.data import DataLoader
5 import math
6 import numpy as np
7 from skimage.metrics import structural_similarity as ssim
8
9 tfm = transforms.Compose([transforms.ToTensor()])
10 train_ds = datasets.MNIST("./data", train=True, download=True, transform=tfm)
11 test_ds = datasets.MNIST("./data", train=False, download=True, transform=tfm)
12 train_dl = DataLoader(train_ds, batch_size=128, shuffle=True)
13 test_dl = DataLoader(test_ds, batch_size=256, shuffle=False)
14
15 def make_blur_kernel(size=9, sigma=2.0):
16     coords = torch.arange(size, dtype=torch.float32)
17     coords -= (size - 1) / 2.
18     g = -(coords**2) / (2 * sigma**2)
19     g = g.view(1, -1) + g.view(-1, 1)
20     g = torch.exp(g)
21     g /= g.sum()
22     return g
23
24 def blur_batch(x, k):
25     C = x.shape[1]
26     k = k.view(1, 1, k.shape[0], k.shape[1]).repeat(C, 1, 1, 1)
27     return F.conv2d(x, k, padding=k.shape[-1]//2, groups=C)
28
29 def psnr(x_hat, x, max_val=1.0):
30     # Calculate Peak Signal-to-Noise Ratio
31     mse = torch.mean((x_hat - x)**2)
32     if mse == 0:
33         return torch.tensor(float('inf'))
34     return 10 * torch.log10((max_val**2) / mse)
35
36 class DeblurCNN(torch.nn.Module):
37     def __init__(self):
38         super().__init__()
39         self.encoder = torch.nn.Sequential(

```

```
40         torch.nn.Conv2d(1, 32, 3, padding=1), # Increase channels, maintain size
41         torch.nn.ReLU(),
42         torch.nn.Conv2d(32, 32, 3, padding=1),
43         torch.nn.ReLU()
44     )
45     self.decoder = torch.nn.Sequential(
46         torch.nn.Conv2d(32, 1, 3, padding=1), # Output 1 channel (image)
47     )
48
49     def forward(self, x):
50         x = self.encoder(x)
51         return self.decoder(x)
52
53 def train_deblurring(model, train_loader, k, lr=1e-3, epochs=5):
54     optimizer = torch.optim.Adam(model.parameters(), lr=lr)
55     criterion = torch.nn.MSELoss() # MSELoss for image reconstruction
56
57     model.train()
58     for ep in range(epochs):
59         running_loss = 0.0
60         for i, (xb, _) in enumerate(train_loader):
61             optimizer.zero_grad()
62             yb = blur_batch(xb, k) # Create blurred input
63             x_hat = model(yb) # Predict deblurred image
64             loss = criterion(x_hat, xb) # Compare prediction to ground truth
65             loss.backward()
66             optimizer.step()
67             running_loss += loss.item()
68             print(f"epoch {ep+1}/{epochs}, loss {running_loss/len(train_loader):.4f}")
69     return model
70
71 def evaluate_deblurring(model, test_loader, k, n_batches=10):
72     model.eval()
73     psnr_blur, psnr_net = 0.0, 0.0
74     ssim_blur, ssim_net = 0.0, 0.0
75
76     with torch.no_grad():
77         for i, (xb, _) in enumerate(test_loader):
78             yb = blur_batch(xb, k)
79             xhat = model(yb).clamp(0.0, 1.0) # Clamp output to valid image range
80
81             # Convert tensors to numpy for skimage.metrics.ssim
82             xb_np = xb.squeeze(1).cpu().numpy() # Remove channel dim
83             yb_np = yb.squeeze(1).cpu().numpy()
84             xhat_np = xhat.squeeze(1).cpu().numpy()
85
```

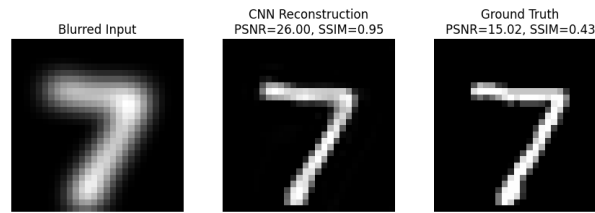


Figure 7: Output for number 7

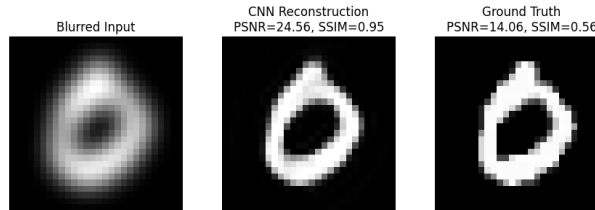


Figure 8: Output for number 0

```

86         # Accumulate PSNR
87         psnr_blur += psnr(yb, xb).item()
88         psnr_net += psnr(xhat, xb).item()
89
90         # Accumulate SSIM for each image in batch
91         for j in range(xb.shape[0]):
92             ssim_blur += ssim(xb_np[j], yb_np[j], data_range=1.0)
93             ssim_net += ssim(xb_np[j], xhat_np[j], data_range=1.0)
94
95         if i+1 >= n_batches:
96             break
97
98         num_samples = n_batches * test_loader.batch_size
99         return psnr_blur/n_batches, psnr_net/n_batches, ssim_blur/num_samples,
100             ssim_net/num_samples
101
102 k = make_blur_kernel(size=9, sigma=2.0)
103 model = DeblurCNN()
104 model = train_deblurring(model, train_dl, k, lr=1e-3, epochs=5)
105
106 p_blur, p_net, s_blur, s_net = evaluate_deblurring(model, test_dl, k, n_batches=10)
107 print("avg PSNR (blur vs GT):", p_blur)
108 print("avg PSNR (CNN vs GT):", p_net)
109 print("avg SSIM (blur vs GT):", s_blur)
110 print("avg SSIM (CNN vs GT):", s_net)

```

The training loss decreases over the five epochs, indicating that the DeblurCNN model is learning to reconstruct clear images from blurred inputs. The final loss of approximately

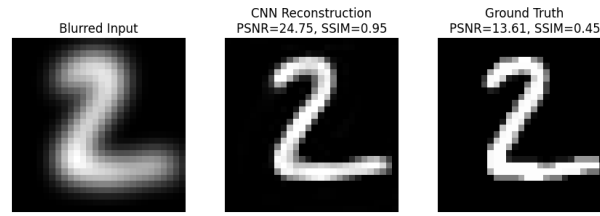


Figure 9: Output for number 2

0.0052 suggests good reconstruction performance.

Table 3: Average Deblurring Evaluation Metrics

Metric	Value
Average PSNR (Blur vs. GT)	14.5401
Average PSNR (CNN vs. GT)	23.4086
Average SSIM (Blur vs. GT)	0.4692
Average SSIM (CNN vs. GT)	0.9300

The CNN significantly improves image quality compared with the blurred input. The PSNR increases from 14.54 to 23.41, indicating that the reconstructed images are substantially closer to the ground truth. Similarly, the SSIM increases from 0.47 to 0.93, showing that the CNN effectively restores structural information.

The visual examples support these results. The blurred MNIST images lose sharpness and fine details, while the CNN reconstructions are noticeably clearer and more closely resemble the original ground truth images. Overall, both the quantitative metrics and visual results demonstrate the effectiveness of the CNN for image deblurring.

6 Guiding Questions

Question 1

Besides standard gradient descent, what other optimization algorithms are commonly used for training neural networks? Briefly describe how they differ conceptually.

SGD with momentum: accumulates a velocity term $v \leftarrow \beta * v - \eta \nabla L$, smoothing updates and accelerating convergence in consistent gradient directions. RMSProp: adapts the learning rate per parameter using a running average of squared gradients. Adam: combines momentum and per-parameter adaptive learning rates using estimates of the first and second moments of the gradient.

They differ conceptually in how they use gradient history: plain GD uses only the current gradient, while momentum/Adam use accumulated past gradients to stabilize and adapt step sizes.

Question 2

Why can a model achieve low training loss but poor validation performance?

Because of overfitting in the model: it memorizes noise/specific patterns of the training dataset instead of learning generalizable structure, so it performs well on seen data but poorly on unseen data.

Question 3

What changes when you replace an MLP with a CNN?

The network exploits spatial structure via local receptive fields and parameter sharing (same kernel across locations), drastically reducing parameters and improving generalization, translation invariance, and suitability for image data, instead of treating pixels as an unstructured flattened vector.

Question 4

What are the feature maps?

Feature maps are the outputs of applying a convolutional kernel across the input (or previous layer's output); each map highlights where a particular learned pattern (e.g., an edge or texture) is present spatially in the image.

7 Conclusions

We reviewed what was covered during the lab, which allowed us to reinforce and acquire new knowledge regarding deep learning and artificial intelligence, covering topics such as linear models, neural networks, convolution, and inverse problems. We were able to demonstrate the correct use of tools for training, validating, and applying various models, along with their potential practical applications.

A key feature of the lab was a natural learning curve that introduced topics sequentially, allowing for a smooth transition to more complex concepts without causing confusion. This was achieved thanks to both the instructions provided and the available documentation on these topics, which made it possible to solve all the proposed problems while learning to generalize the concepts for potential future applications.

Knowledge in the area of inverse problems—a topic covered in the previous lab—is reinforced, since their application in the field of image processing was demonstrated; we acquire the ability to train MLPs and understand their components as well as how they work; we build a simple CNN with the goal of restoring images, which gives us a more tangible understanding of what inverse problems mean in practice; and finally, we revisit the topic of evaluation metrics, which serve to complement the work done in training models.

On a personal note, I must admit that I was very satisfied with the methodology used, as I was able to understand the theory before, during, and after its application thanks to the ease of finding documentation on this topic. And for me, this is another step toward my

personal goal of mastering these concepts. Every new discovery fuels the curiosity inherent in human beings, and I eagerly look forward to becoming part of the academic community to contribute to such a rewarding source of knowledge.

References

- [1] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, p. 436, 2015, doi: 10.1038/nature14539.
- [2] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, p. 600, 2004, doi: 10.1109/TIP.2003.819861.
- [3] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015, Springer*, vol. 9351, p. 234, 2015, doi: 10.1007/978-3-319-24574-4_28.
- [4] G. Ongie, A. Jalal, C. A. Metzler, R. G. Baraniuk, A. G. Dimakis, and R. Willett, “Deep learning techniques for inverse problems in imaging,” *IEEE Journal on Selected Areas in Information Theory*, vol. 1, no. 1, p. 39, 2020, doi: 10.1109/JSAIT.2020.2991563.
- [5] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” 2014. [Online]. Available: <https://jmlr.org/papers/v15/srivastava14a.html>